

GenLimitLib: Theoretical Findings and Written Proofs

September 25, 2026

4 Theoretical Findings

We present three mathematical case studies developed while building and working with GenLimitLib: a proof repair, a characterization of proper generation under replay, and an exact mistake–convergence tradeoff for a special family. The appendix gives the research workflow and complete written proofs; the accompanying repository contains their Lean verification.

4.1 Identifying and repairing a proof gap

In our first example we identify a gap in the proof of a published generation theorem and give a short repair. The repair preserves the original algorithm and theorem statement.

In *Pareto-optimal Non-uniform Language Generation*, Charikar and Pabbaraju (2026, P13) construct a generator whose convergence guarantee depends on the target language but holds for every presentation order. Their proof of Theorem 8 uses a procedure that inserts languages into an ordered list. When a language L is inserted, the procedure considers subcollections containing L , drawn from the list up to L , whose intersection is finite. It stores the largest such intersection size as a score $m^*(L)$, together with a subcollection $\mathcal{C}(L)$ attaining it. If no such subcollection exists, it stores zero and an empty subcollection. Claim 7 asserts that the stored subcollection remains a maximizer as further languages are inserted.

While formalizing Claim 7, GenLimitLib records a counterexample. Take two disjoint infinite languages A and B . When A is inserted first, no subcollection containing A has finite intersection, so the procedure stores $\mathcal{C}(A) = \emptyset$ and $m^*(A) = 0$. When B is inserted, $\{A, B\}$ has intersection size zero, and the insertion rule places B before A . However, the stored subcollection $\mathcal{C}(A)$ remains empty. It does not contain A , so it cannot be the maximizing subcollection asserted by Claim 7.

Our repair proves that every subcollection containing L , drawn from the current list up to L and having finite intersection, has intersection size at most the stored score $m^*(L)$. It proves that this bound is preserved after each insertion. This is the bound needed to complete the generation proof.

4.2 Connecting papers to derive new results

GenLimitLib also supports transferring constructions between different learning problems. We connect a bounded-memory example from P31 to the replay model of P22, obtaining a stronger impossibility example. This connection then motivates an exact characterization result.

Language Generation with Replay (Racca et al., 2026, P22) studies a model in which the adversary can use the learner’s earlier outputs to contaminate future observations. The paper studies several generation guarantees in this setting. We focus on *proper generation in the limit*: proposals must come from a fixed family and eventually be contained in the true target.

A different paper, *On Language Generation in the Limit with Bounded Memory* (Kleinberg et al., 2026a, P31), asks how restricting a learner’s memory affects generation. It gives a three-language example on which an incremental index-based generator, retaining only its previous proposed index between observations, cannot succeed. The languages are $T \cup \{a, b\}$, $T \cup \{a, c\}$, and $T \cup \{b, c\}$, where T is infinite and a, b, c are distinct points outside T .

Using GenLimitLib, we show that this same example also makes deterministic proper generation under replay impossible, even with unrestricted memory. Our proof combines P31’s construction with the indistinguishability argument in P22’s impossibility proof. This improves P22’s four-language impossibility example to three languages.

This connection motivates a broader question: which finite families permit proper generation under replay? Our Theorem 1 below gives an exact characterization. We impose no computability restriction on the generator.

Theorem 1 (When proper generation under replay is possible). *Let $L_1, \dots, L_N$, with $N \geq 1$, be a finite family of infinite countable languages. Deterministic proper generation under replay is possible if and only if, for every possible first observation x , there is a first proposal L_j with the following property. Group the indices i with $x \in L_i$ according to equality of $L_i \setminus L_j$. For every such group E , some language L_h from the original family satisfies $L_h \subseteq \bigcap_{i \in E} L_i$.*

4.3 Resolving an open problem in a special case

Mistake-Bounded Language Generation (Kleinberg et al., 2026b, P29) studies generators that learn from positive examples and output an unobserved point at each step. A mistake occurs when the output is outside the target language. The paper asks how the total number of mistakes trades off against how quickly the generator stops making them (Section 8, Open Direction (2)). We resolve this question for deterministic generation on the family underlying the paper’s tradeoff construction. GenLimitLib’s separate definitions of mistake and convergence guarantees helped guide this analysis. As in P29, observations are distinct, and time counts the points observed so far.

We consider the family of languages $L_i = P_i \cup R_i$, where $P_i = \bigcup_{k \leq i} C_k$. The blocks C_i are nonempty and finite, the sets R_i are countably infinite, and all these sets are mutually disjoint. Thus each successive language contains one more finite block, together with an infinite part R_i that belongs only to that language. We call this the *staircase family* and write $s_i = |P_i|$. We measure convergence by the number of distinct observations needed before all later outputs are guaranteed to be valid, for every presentation order.

For this family, the tradeoff comes from a simple choice. When the observed set is exactly P_k , the generator can output in R_k , which is valid only for L_k , or in C_{k+1} , which is valid for every later target. Away from these points, it can choose an unobserved point common to all compatible targets. Let $\sigma_k = 1$ record the choice of R_k and $\sigma_k = 0$ the choice of C_{k+1} . Write $M_i(G)$ for the worst-case number of mistakes of a generator G on L_i , and $D_i(G)$ for its worst-case convergence time.

Theorem 2 (Optimal mistake and convergence guarantees). *For a binary sequence $\sigma = (\sigma_k)_{k \geq 1}$, let $J_i(\sigma) = \{1 \leq k < i : \sigma_k = 1\} \cup \{i : \sigma_i = 0\}$ and define*

$$m_i(\sigma) = |J_i(\sigma)|, \quad d_i(\sigma) = \max(\{0\} \cup \{s_k + 1 : k \in J_i(\sigma)\}).$$

Each sequence σ is realized by a deterministic generator G_σ with $M_i(G_\sigma) = m_i(\sigma)$ and $D_i(G_\sigma) = d_i(\sigma)$ for all i . Conversely, every deterministic generator G admits a sequence σ such that $m_i(\sigma) \leq$

$M_i(G)$ and $d_i(\sigma) \leq D_i(G)$ for every i . These are exactly the Pareto-optimal guarantees: none can be improved without worsening another.

The theorem gives matching constructions and lower bounds for every optimal tradeoff on this family. It determines both the number and the latest possible timing of mistakes, for arbitrary positive finite block sizes (Appendix A.4).

A Theoretical Findings: Research Workflow and Proofs

This appendix first describes the research design and workflow used to develop and check the findings in Section 4. Complete written proofs follow in the order of the three main-text cases.

A.1 Research design and workflow

We supplied advanced reasoning models, including GPT-6 Astra with Ultra reasoning and GPT-6 Pro, with the Lean library, paper maps, and relevant source manuscripts. The models compared definitions and assumptions across papers, looked for connections and simpler arguments, and proposed new questions. Promising directions were developed through successive rounds of statement refinement, proof construction, example testing, and review. The number of iterations and the supplied material varied with the question.

The workflow combined mathematical exploration with focused checks. Follow-up sessions examined proofs, searched for counterexamples, ran finite checks, and formalized selected statements in Lean. For each featured finding, the following proof subsections identify its source material, additional mathematical contribution, written proof, and Lean verification status.

Overview of proofs and verification. Table 1 locates each proof and distinguishes the written arguments and their Lean verification.

Table 1: Proofs of the featured findings and scope of formal verification.

Finding	Proof	Lean scope
Section 4.1: local invariant repair	Appendix A.2	Counterexample, corrected score bound, and the repaired generation guarantee, including repeated observations.
Theorem 1: finite proper replay	Appendix A.3	Three-language impossibility, both directions of the characterization, and finite-state consequences.
Theorem 2: exact staircase frontier	Appendix A.4	Exact guarantees, lower bounds for arbitrary deterministic generators, and Pareto optimality.

Proof materials. The complete Lean sources, theorem-to-code correspondence, and reproducible verification commands accompany these written proofs in the anonymous artifact repository. The formalization concerns the deterministic mathematical models stated below; it does not assert effective oracle implementations or running-time bounds.

A.2 Identifying and repairing a proof gap in Pareto-optimal generation

In this subsection we give a repair to the proof of the generation-time theorem of Charikar and Pabbaraju (2026, P13). During formalization we found that published Claim 7 (arXiv Claim 3.2),

used in published Theorem 8 (arXiv Theorem 4), can fail when Procedure 1 stores an empty subcollection to indicate that no candidate exists. We explain the claim’s role, exhibit the failure, and prove the numerical bound needed to complete the generation argument. The repaired proof preserves the original algorithm and its generation-time guarantee. References below use the published ALT 2026 numbering.

A.2.1 The insertion procedure and the original claim

Let $L_1, L_2, \dots$ be infinite languages over a countable universe U . Procedure 1 inserts these languages one at a time into a finite ordered list. It assigns each inserted language L_i a nonnegative integer m_i and a stored subcollection C_i ; both remain fixed after that insertion. Here m_i denotes the source paper’s $m^*(L_i)$. The order, and hence the languages preceding L_i , can change when later languages are inserted.

Identify a subcollection with its set of indices. For a finite index set S , write

$$I(D) = \bigcap_{j \in D} L_j, \quad \mathcal{C}_i(S) = \{D \subseteq S : i \in D \text{ and } I(D) \text{ is finite}\}.$$

Thus a member of $\mathcal{C}_i(S)$ is a subcollection containing L_i whose intersection is finite. Define

$$M_i(S) = \max(\{0\} \cup \{|I(D)| : D \in \mathcal{C}_i(S)\}).$$

The maximum exists because S has finitely many subcollections. If $\mathcal{C}_i(S)$ is nonempty, it is the largest finite intersection size attained by a candidate. If the candidate set is empty, its value is zero by convention. In particular, $M_i(S) = 0$ does not distinguish an empty candidate set from a nonempty candidate set whose intersections are all empty.

Suppose a new language L_q is immediately to the right of L_i during its insertion, and let S_i consist of i and all indices preceding i in the old order. The procedure computes

$$m_{\text{check}} = M_q(S_i \cup \{q\})$$

and moves q to the left of i if $m_{\text{check}} \leq m_i$. It repeats this comparison while moving leftward. The checked maximum is recomputed in the relevant prefix; it need not equal the score eventually assigned to q . Once the insertion ends, let S_q be the prefix through q . The procedure sets $m_q = M_q(S_q)$ and chooses $C_q \in \mathcal{C}_q(S_q)$ attaining this maximum if a candidate exists. Otherwise it sets $C_q = \emptyset$ and $m_q = 0$.

Claim 7 asserts that the stored subcollection for an old language continues to be a maximizing candidate in its current prefix. In the notation above, its maximizing-witness assertion is

$$C_i \in \arg \max_{D \in \mathcal{C}_i(S_i)} |I(D)|.$$

The proof of Theorem 8 uses this assertion to bound the size of a finite intersection involving L_i . Specifically, the generator scans the ordered languages, retaining a language when it contains every observed point and its addition leaves the retained languages with an infinite intersection. To show that the true target L_i is retained after more than m_i distinct observations, the proof needs the implication

$$D \in \mathcal{C}_i(S_i) \implies |I(D)| \leq m_i.$$

The assertion that the particular stored subcollection C_i attains the maximum is stronger than this implication.

A.2.2 Failure of the maximizing-witness assertion

Let A and B be disjoint infinite languages, inserted in that order. At the first insertion, the only subcollection containing A is $\{A\}$, whose intersection is infinite. There is therefore no candidate, and the procedure stores $C_A = \emptyset$ and $m_A = 0$.

When B is inserted, the subcollection $\{A, B\}$ has empty intersection. It is a candidate for B with score zero, whereas $\{B\}$ still has infinite intersection. Consequently the checked maximum is zero. The comparison $0 \leq m_A = 0$ moves B before A , giving the order (B, A) . The prefix through A now contains both languages, and $\{A, B\}$ is a candidate for A attaining the maximum zero. However, the stored subcollection for A is still $\emptyset$. It does not contain A , so it is not a candidate and cannot belong to the displayed argmax.

The failure is the identification of an empty stored subcollection with a maximizing candidate. The empty subcollection contains no languages; the candidate $\{A, B\}$ contains two languages whose intersection contains no points. Assigning score zero in both cases does not make the two subcollections interchangeable. The numerical inequality needed by Theorem 8 remains true in this execution: every candidate in the prefix through A has intersection size at most $m_A = 0$. The published induction controls the maximum value, but this alone does not ensure that the stored subcollection is still a candidate.

A.2.3 The corrected statement and its preservation

The repair replaces the assertion about a stored maximizing subcollection by an upper bound on every candidate's intersection size. The following lemma provides the induction step: it shows that the numerical bound persists when a new language enters an old language's prefix.

Lemma A.2.1 (Preservation of the score bound). *Let S be a finite set of indices and let $b \in \mathbb{N}$. Suppose that every $D \in \mathcal{C}_i(S)$ satisfies $|I(D)| \leq b$. If $q \notin S$ and $M_q(S \cup \{q\}) \leq b$, then every $D \in \mathcal{C}_i(S \cup \{q\})$ also satisfies $|I(D)| \leq b$.*

Proof. Fix $D \in \mathcal{C}_i(S \cup \{q\})$. There are two cases. If $q \notin D$, then $D \subseteq S$, and it still contains i and has finite intersection. Thus $D \in \mathcal{C}_i(S)$, so the assumed bound gives $|I(D)| \leq b$.

If $q \in D$, then $D \subseteq S \cup \{q\}$ contains q and has finite intersection. It is therefore also a candidate in the optimization defining $M_q(S \cup \{q\})$. Hence

$$|I(D)| \leq M_q(S \cup \{q\}) \leq b.$$

Both cases establish the required inequality. The argument includes $b = 0$ and does not require a stored maximizing subcollection. $\square$

Proposition A.2.2 (Corrected form of Claim 7). *At every completed insertion stage, let L_i be an inserted language, let m_i be its assigned score, and let S_i be its current prefix, including i . Then*

$$\text{for every } D \subseteq S_i, \quad i \in D \text{ and } I(D) \text{ finite} \implies |I(D)| \leq m_i. \quad (1)$$

Proof. We induct on the number of insertions. The assertion is vacuous before any language is inserted. Suppose it holds for the old order and insert L_q . For the new language, its assigned score is $m_q = M_q(S_q)$, so the bound follows directly from the definition of the maximum in its final prefix S_q . In particular, it is vacuous when there is no candidate and remains valid when the maximum is zero.

Consider an old language L_i . If q does not move to the left of i , its prefix and assigned score remain unchanged, and the inductive hypothesis applies. If q moves to the left of i , let S_i be its old

prefix. The relative order of all old languages is unchanged, so its new prefix is exactly $S_i \cup \{q\}$. When q crosses i , the procedure's comparison gives

$$M_q(S_i \cup \{q\}) \leq m_i.$$

The inductive hypothesis bounds every i -candidate in S_i by m_i . Applying Lemma A.2.1 with $b = m_i$ proves the bound in its new prefix. Subsequent movements of q farther to the left change neither this set of indices nor m_i . This covers every old language and completes the induction. $\square$

A.2.4 Recovery of the generation-time guarantee

For completeness, we give the step connecting the corrected bound to the original generation algorithm. Let $H \subseteq L_i$ be the finite set of distinct observations from the target L_i , and suppose that i is included in the finite list currently being scanned and that $|H| \geq m_i + 1$. Let B be the indices retained before the scan reaches i . Every retained language contains H , and every index in B precedes i . Consequently

$$B \cup \{i\} \subseteq S_i, \quad H \subseteq I(B \cup \{i\}).$$

The target satisfies the sample-consistency test because $H \subseteq L_i$. If the scan rejected it, its intersection with the previously retained languages would be finite. Then $B \cup \{i\}$ would be an i -candidate, and (1) would imply

$$m_i + 1 \leq |H| \leq |I(B \cup \{i\})| \leq m_i,$$

a contradiction. Thus the scan retains L_i .

The scan never removes a retained language and only adds a language when the common intersection remains infinite. Its final intersection is therefore infinite and, because it includes L_i among the intersected languages, is a subset of L_i . Removing the finite observed set leaves a point in $L_i \setminus H$. The original generator's choice of such a point is consequently a fresh target output.

In Theorem 8, the generator considers the first $f(r)$ languages after r observations, where f is nondecreasing and unbounded. Observations may repeat; let H_r be the set of distinct points observed by round r . Write $g(i) = \min\{r : f(r) \geq i\}$ for the first round at which L_i is included. If

$$|H_r| \geq t_i := \max\{g(i), m_i + 1\},$$

then $r \geq |H_r| \geq g(i)$, so $f(r) \geq i$, and $|H_r| > m_i$. The target is therefore under consideration and the preceding argument guarantees a fresh target output. This proves the same bound in terms of distinct observations as in the source paper, including presentations with repetitions. The score assignments, insertion comparisons, greedy scan, and output rule all remain unchanged. Only the intermediate proof assertion is replaced.

Formal verification. GenLimitLib checks the two-language counterexample, the corrected score bound through successive insertions, target retention, and the resulting generation-time guarantee. The accompanying Lean development also proves the version above with repeated observations, retaining the original scheduler based on the total number of rounds. The repository maps these statements to P13's existing modules and the additional proof in `Section4/InvariantRepair.lean`.

A.3 Proper generation under replay

This subsection proves the three-language impossibility discussed in Section 4.2 and the finite-family characterization in Theorem 1. We first specify the replay model, then explain why the three-language construction prevents successful generation, and finally identify exactly when a finite family admits a successful learner. The model follows Racca et al. (2026, P22); the three-language family comes from Kleinberg et al. (2026a, P31). The Lean development verifies the transferred impossibility, both directions of the characterization, and the finite-state consequences proved below.

A.3.1 The replay model

Let $\mathcal{H} = (L_1, \dots, L_N)$, with $N \geq 1$, be a finite family of infinite subsets of a countable universe U , and let $X = \bigcup_{i=1}^N L_i$. Different indices may name the same language. The target is one fixed member L_i of the family. At round $n \geq 0$, the learner first receives a point $x_n \in X$ and then outputs an index $j_n \in [N]$. Thus a proper learner always proposes one of the languages in $\mathcal{H}$, although that language need not contain every point observed so far.

A *complete replay text* for target L_i satisfies

$$x_n \in L_i \cup \bigcup_{t < n} L_{j_t} \quad (n \geq 0), \quad (2)$$

$$L_i \subseteq \{x_n : n \geq 0\}. \quad (3)$$

The first condition allows each observation to come either from the target or from any language previously proposed by the learner. The second condition requires every target point eventually to appear. Observations may repeat, and there is no bound on how long the adversary may delay a particular target point. Because there is no earlier output at round zero, the first observation satisfies $x_0 \in L_i$.

The learner succeeds if $L_{j_n} \subseteq L_i$ for all sufficiently large n . It need not identify the target's index or output the whole target language. We initially allow any deterministic learner that uses its full input history. No computability restriction is imposed on the family or on the learner. The finite-state construction below will use only the membership profile of each observation, namely the set of languages that contain it.

A.3.2 The three-language impossibility

The construction from the bounded-memory paper gives a useful first example of what replay can hide. After the first output, the same complete input stream can be legal for two targets that have no common allowable output. The argument applies to learners with full history.

Proposition A.3.1 (Three-language impossibility). *Let $T \subseteq U$ be infinite, and let $a_1, a_2, a_3 \in U \setminus T$ be distinct. The family $L_i = T \cup \{a_s : s \neq i\}$, for $i \in \{1, 2, 3\}$, does not admit deterministic proper generation in the limit with replay.*

Proof. Suppose that a learner succeeds for this family. Give it a first point $x_0 \in T$, and let its first output be j . Write p and q for the two indices different from j . Continue the input by enumerating every point of $S = T \cup \{a_1, a_2, a_3\}$.

This is a complete legal replay text for either fixed target L_p or fixed target L_q . Indeed, the first point belongs to both targets, and $S = L_p \cup L_j = L_q \cup L_j$. Every subsequent point is therefore either a target point or a point of the first output language L_j , which is already eligible for replay.

The continuation also contains every point of each target. Legality uses only the first output; it places no restriction on the learner's later choices.

The learner sees the same input history in the two cases, so it produces the same output sequence. If it succeeded for both targets, then after the larger of the two success times every output language would be contained in $L_p \cap L_q = T \cup \{a_j\}$. But every language in the family contains two of the three exceptional points, so none is contained in this intersection. This is a contradiction. $\square$

A.3.3 The finite-family characterization

The preceding proof identifies the relevant ambiguity: two targets can have the same union with the learner's first output language. Equivalently, they agree outside that language. We now formulate a condition requiring all targets that remain indistinguishable in this way to admit a common allowable output.

For $x \in X$, define its membership profile by $\text{prof}(x) = \{i \in [N] : x \in L_i\}$, and let $\mathcal{P} = \{\text{prof}(x) : x \in X\}$. Every $P \in \mathcal{P}$ is nonempty. If the first observation has profile P , precisely the indices in P are possible targets at that point. For a proposed first output j , write $R_j(i) = L_i \setminus L_j$ for the part of target L_i outside that output. For each $i \in P$, define

$$E_j(i; P) = \{k \in P : R_j(k) = R_j(i)\}.$$

This is the group of possible targets having the same residual as L_i . The groups partition P . A common allowable output for such a group is an index $h \in [N]$ satisfying $L_h \subseteq L_k$ for every $k \in E_j(i; P)$. The language L_h need not equal their intersection, and its index need not belong to the group.

Theorem A.3.2 (Exact finite characterization). *The family $\mathcal{H}$ admits deterministic proper generation in the limit with replay if and only if the following condition holds: for every $P \in \mathcal{P}$, there is an index $j \in [N]$ such that, for every $i \in P$, there is an index $h \in [N]$ with*

$$L_h \subseteq \bigcap_{k \in E_j(i; P)} L_k.$$

The index j need not belong to P .

Proof. Necessity. Fix a realized profile P and choose a first point x with this profile. Let j be the first output of a learner that succeeds for every target and every complete legal replay text. Consider any group $E = E_j(i; P)$, and let R denote the residual shared by its members. After x , supply an enumeration of $S = L_j \cup R$.

For each $k \in E$, we have $x \in L_k$ and $S = L_j \cup L_k$. Consequently the continuation is legal for target L_k , because the first output permits replay from L_j , and it is complete because it enumerates all of L_k . These are different possible fixed targets for one and the same input sequence. The learner's outputs are therefore identical in all cases.

By success on target L_k , all sufficiently late outputs must be contained in L_k . There are only finitely many indices in E , so we can take the largest of their success times. Every output after that time is a member of $\mathcal{H}$ contained in every L_k with $k \in E$. At least one such output exists, giving the required index h . Since this argument applies to every group, the first output j satisfies the condition for P .

Sufficiency. Assume the stated condition. For each realized profile P , fix an index $j(P)$ satisfying it. Also, for every nonempty set of indices $E \subseteq [N]$ for which a common allowable output

exists, fix one such index $h(E)$. These choices are made once for the whole family. In particular, $h(E)$ depends only on E , not on the history by which the learner reaches that set.

On receiving its first observation, the learner sets $P = \text{prof}(x_0)$, outputs $j = j(P)$, and initializes the candidate set $C = P$. It keeps the index j for the rest of the run. For every subsequent observation x , it proceeds as follows:

1. If $x \notin L_j$, replace C by $\{k \in C : x \in L_k\}$. If $x \in L_j$, leave C unchanged.
2. Examine the residuals $R_j(k)$ for $k \in C$. If one of them is contained in every other surviving residual, let R be this least residual and let $E = \{k \in C : R_j(k) = R\}$. Output $h(E)$.
3. If there is no least residual, output j .

On an empty candidate set the learner may output j ; we will show that this case never occurs on a legal replay text.

The distinction between a *least* residual and an *inclusion-minimal* residual is essential here. A least residual is contained in every surviving residual. An inclusion-minimal one merely has no strictly smaller survivor; other survivors may be incomparable with it. The proof needs containment in the true residual, so the rule uses a least residual and otherwise returns to j .

We first justify that $h(E)$ in the rule is defined. Initially $C = P$. An update tests membership only at a point outside L_j . Two indices with equal residuals give the same answer to every such test. Thus an update either retains an entire original group $E_j(i; P)$ or removes the entire group. By induction, C is always a union of these original groups. If a least surviving residual exists, its indices form one entire original group, for which the assumed condition provides a common allowable output.

We next prove the property that makes the updates reliable. Fix the true target i_* . At every stage, the candidate set contains i_* , and every output language lies inside $L_j \cup L_{i_*}$. Both assertions hold initially: $x_0 \in L_{i_*}$ gives $i_* \in P$, and the first output is j . Suppose they hold for all earlier outputs, and consider the next observation x . By replay legality, x belongs either to L_{i_*} or to an earlier output language. Every such earlier language lies in $L_j \cup L_{i_*}$ by the induction hypothesis. Therefore, if $x \notin L_j$, necessarily $x \in L_{i_*}$. The update retains the true target, so C stays nonempty.

It remains to check the new output. Outputting j preserves the claimed containment immediately. Otherwise the learner outputs $h(E)$ for a least residual R . Choose $k \in E$. Since $L_{h(E)} \subseteq L_k$ and the true target is still in C , we have

$$L_{h(E)} \setminus L_j \subseteq L_k \setminus L_j = R \subseteq L_{i_*} \setminus L_j.$$

Hence $L_{h(E)} \subseteq L_j \cup L_{i_*}$, as required. This completes the induction. In particular, the construction satisfies

$$L_{j_n} \setminus L_{i_*} \subseteq L_j \quad (n \geq 0). \tag{4}$$

Thus any point outside the target that an output makes available for replay was already available from the first output. The proof establishes that later observations outside L_j are trustworthy; it does not assume this when defining the candidate updates.

Finally, we prove convergence. For every initial candidate $k \in P$ such that $R_j(i_*) \not\subseteq R_j(k)$, choose a point $w_k \in R_j(i_*) \setminus R_j(k)$. This point belongs to the true target and lies outside L_j , but does not belong to L_k . Completeness guarantees that it eventually appears, at which point k is removed if it has not been removed already. There are only finitely many initial candidates, so there is a time after all these witnesses have appeared. From then on, every surviving residual contains $R_j(i_*)$. Since i_* survives, its residual is a least surviving residual.

Moreover, every initial candidate with residual equal to $R_j(i_*)$ survives forever: each observation used for an update is a true-target point outside L_j , and hence belongs to every one of these equal residuals. The group selected by the learner from that time onward is therefore the fixed set $E_j(i_*; P)$. Its chosen output $h(E_j(i_*; P))$ is contained in L_{i_*} , because i_* is a member of the group. The learner stabilizes to this correct index, proving sufficiency. $\square$

The proof also gives quantitative information about the learner and shows that outputs can be restricted to inclusion-minimal languages in the family. We state these consequences separately to distinguish them from the existence criterion.

Corollary A.3.3 (Finite memory and output choices). *Whenever the condition of Theorem A.3.2 holds, there is a learner that reads only membership profiles, uses at most $1 + N2^N$ states, changes its output index at most N times, and stabilizes to a correct index. It can be chosen so that every output language is inclusion-minimal in $\mathcal{H}$, meaning that no language in $\mathcal{H}$ is its proper subset. The containment property (4) continues to hold.*

Proof. For the state bound, there is one initial state before the first observation. Thereafter the learner need only store (j, C) , where $j \in [N]$ and $C \subseteq [N]$. The update tests $j \in \text{prof}(x)$ and, when this fails, replaces C by $C \cap \text{prof}(x)$. All residual comparisons and the choices $j(P)$ and $h(E)$ are fixed data of the finite family. Thus a finite-state transducer can implement the rule with at most $1 + N2^N$ states. This is an existence statement for the fixed family, rather than an assertion that an arbitrary representation allows these data to be computed.

After the first output, the output is a fixed function of (j, C) . Consequently it can change only when C strictly decreases, apart from a possible change on the first subsequent round when the initial output j is replaced by the choice for the same candidate set. Because C starts nonempty and always contains the true target, it strictly decreases at most $|P| - 1 \leq N - 1$ times. There are therefore at most N output changes. Stabilization and containment were proved in the theorem.

To obtain inclusion-minimal outputs, first consider any index j that satisfies the theorem's condition for a profile P . Since the family is finite, there is an inclusion-minimal member $L_{j'}$ of the family with $L_{j'} \subseteq L_j$. If two targets agree outside $L_{j'}$, they also agree outside L_j , because $U \setminus L_j \subseteq U \setminus L_{j'}$. Each group formed using j' is therefore contained in a group formed using j . A language contained in every target of the larger group is also contained in every target of the smaller one. Thus j' satisfies the same condition, and every first-output choice can be made inclusion-minimal.

Likewise, whenever a group E admits a common allowable language, choose an inclusion-minimal member of the family below that language. It is still contained in every target in E . Fix these minimal choices globally as the selectors $h(E)$ and run the same construction. Every output is now inclusion-minimal, while the state bound, output-change bound, convergence proof, and containment argument remain valid. $\square$

A.4 Optimal mistake and convergence guarantees for the staircase family

In this subsection we prove Theorem 2, which determines the optimal mistake and convergence guarantees for the staircase family. This family generalizes the construction in Theorem 6.4 of Kleinberg et al. (2026b, P29) by allowing arbitrary positive finite block sizes. We compare generators on every possible target simultaneously: improving one target's guarantee may worsen another's. The proof constructs generators from a sequence of binary choices, derives matching lower bounds for arbitrary generators, and shows that every resulting choice expresses an optimal tradeoff.

A.4.1 Languages, histories, and guarantees

We first give a concrete realization of the family in Section 4.3. Fix positive integers $a_1, a_2, \dots$. On $X = \mathbb{N}_{>0} \times \mathbb{N}_{>0}$, put

$$C_i = \{(i, j) : 1 \leq j \leq a_i\}, \quad P_i = \bigcup_{k=1}^i C_k, \quad s_i = |P_i| = \sum_{k=1}^i a_k, \quad s_0 = 0.$$

Write $R_i = \{(i, j) : j > a_i\}$ for the private remainder of column i , and define

$$L_i = P_i \cup R_i = P_{i-1} \cup (\{i\} \times \mathbb{N}_{>0}).$$

Each L_i is infinite. For $k < i$, its intersection with L_k is P_k . A point of C_k belongs to exactly the languages L_i with $i \geq k$, whereas a point of R_k belongs only to L_k . For the abstract family in the main text, choose a bijection from each finite block to the first a_i points of its column and from its countably infinite private part to the remainder. These bijections preserve every language membership. Any additional ambient points lie in no target; the lower-bound argument below also allows a generator to output them. Thus the concrete notation entails no restriction on the stated family.

A legal history for target L_i is a finite ordered list of distinct points in L_i . After a history of length $t \geq 0$, a generator G outputs a point outside its observed set, before the next observation arrives. A generator is a deterministic function of the entire ordered history; no computability restriction is imposed. An output is a mistake when it lies outside the fixed target. This definition imposes freshness relative to observed inputs, without requiring the outputs to be mutually distinct.

Let $M_i(G) \in \mathbb{N} \cup \{\infty\}$ be the supremum of the total number of mistakes over injective positive streams in L_i . Let $D_i(G) \in \mathbb{N} \cup \{\infty\}$ be the least nonnegative integer D such that G is valid on every legal L_i -history of every length $t \geq D$; set $D_i(G) = \infty$ if no such finite D exists. In particular, an error after exactly s observations forces $D_i(G) \geq s + 1$, and $D_i(G) = 0$ means that no legal history receives an invalid output. Time is the number of distinct observations, including time zero at the empty history.

The same worst-case quantities result if streams are required to enumerate the entire target. Every finite injective positive history extends to such an enumeration. Thus every finite collection of mistakes witnessed on a positive stream is also witnessed on some complete text, and every history used to test a convergence time occurs on a complete text.

The two quantities answer different questions: $M_i(G)$ bounds how many errors can occur, while $D_i(G)$ bounds how late an error can occur. We compare both quantities for all targets at once. Specifically, G' dominates G if $M_i(G') \leq M_i(G)$ and $D_i(G') \leq D_i(G)$ for every i . A profile is Pareto minimal if no generator dominates it with a strict inequality in at least one coordinate. Thus a Pareto-minimal profile cannot be improved on any target without worsening at least one of the guarantees.

A.4.2 Characterization and proof

The relevant decisions occur when the observed set is exactly P_k . At that point the possible targets are $L_k, L_{k+1}, \dots$, whose intersection is P_k , so no unobserved point is valid for all of them. Choosing a point in R_k is correct only for L_k ; choosing a point in C_{k+1} is correct for every later target. We record these choices by a binary sequence $\sigma = (\sigma_k)_{k \geq 1}$, with $\sigma_k = 1$ for the first choice and $\sigma_k = 0$ for the second. The construction below makes no mistakes away from these decision points.

For target L_i , an earlier choice of R_k , with $k < i$, causes one mistake if the observed set reaches P_k . The choice of C_{i+1} causes a mistake at P_i . These observations motivate the candidate mistake bound and the corresponding set of possible error times:

$$m_i(\sigma) = \sum_{k < i} \sigma_k + 1 - \sigma_i, \quad E_i(\sigma) = \{s_k : k < i, \sigma_k = 1\} \cup \{s_i : \sigma_i = 0\}.$$

The proposed convergence time is one more than the latest of these times, or zero if there are none:

$$d_i(\sigma) = \begin{cases} 0, & E_i(\sigma) = \emptyset, \\ 1 + \max E_i(\sigma), & E_i(\sigma) \neq \emptyset. \end{cases} \quad (5)$$

The first part of the theorem shows that these formulas are attained. The second proves that every generator has guarantees at least as large as those of one such sequence, simultaneously for every target. The third establishes that none of the resulting profiles can be improved further.

Theorem A.4.1 (Optimal mistake and convergence guarantees). *For the staircase family above, the following assertions hold.*

1. *For every binary sequence σ , there is a deterministic generator G_σ satisfying, simultaneously for all $i \geq 1$,*

$$M_i(G_\sigma) = m_i(\sigma), \quad D_i(G_\sigma) = d_i(\sigma). \quad (6)$$

2. *Every deterministic generator G is dominated by a schedule: there is one σ such that, for all $i \geq 1$,*

$$m_i(\sigma) \leq M_i(G), \quad d_i(\sigma) \leq D_i(G). \quad (7)$$

3. *Every schedule profile is Pareto minimal. Consequently the profiles in (6) are exactly the deterministic Pareto frontier.*

Proof. Attaining the proposed guarantees. We first define G_σ on every finite history and then prove the two equalities in (6). Choose fixed orders for making all the selections below. If a history contains a point of R_i , any compatible target must be L_i . At such a realizable history, output any unobserved point of L_i . If a history is unrealizable, output any unobserved point of X ; this branch is irrelevant to targetwise guarantees but makes the generator total and fresh.

It remains to define the generator before a private point appears. At the empty history, output a point of C_1 , which belongs to every target. At a nonempty history consisting only of finite-block points, let k be the largest observed column index and let S be its observed set. The compatible targets are precisely $L_k, L_{k+1}, \dots$, and their intersection is P_k . Indeed, the observed point of C_k excludes every earlier target, while every target of index at least k contains all the observed points. Their common core is P_k because $L_k \cap L_{k+1} = P_k$.

If $S \neq P_k$, output a point of $P_k \setminus S$. It is fresh and valid for every compatible target. If $S = P_k$, no fresh point is valid for all compatible targets. When $\sigma_k = 1$, output a point of R_k , valid only for L_k . When $\sigma_k = 0$, output a point of C_{k+1} , valid for every later target and invalid for L_k . Both choices are fresh.

Fix a target L_i . Its stream can encounter a boundary $S = P_k$ only for $k \leq i$, and it can encounter each such boundary at most once because observations are injective. Every output away from these boundaries is valid. At a boundary $k < i$, a mistake occurs exactly when $\sigma_k = 1$; at boundary $k = i$, a mistake occurs exactly when $\sigma_i = 0$. The respective observed counts are s_k . This proves $M_i(G_\sigma) \leq m_i(\sigma)$ and $D_i(G_\sigma) \leq d_i(\sigma)$.

To attain these bounds, present $C_1, C_2, \dots, C_i$ in fixed internal orders, followed by an enumeration of R_i . This is an injective complete text of L_i , and it encounters every boundary P_k for $k \leq i$. It therefore realizes exactly the mistakes in $E_i(\sigma)$. If that set is nonempty, its latest mistake occurs after $\max E_i(\sigma)$ observations, forcing convergence time at least $1 + \max E_i(\sigma)$. If it is empty, the preceding upper bound already gives convergence time zero. This establishes both equalities in (6).

Lower bounds for an arbitrary generator. We next show that allowing more elaborate choices, including dependence on the order of observations, cannot improve on all the profiles above. Fix a deterministic fresh generator G . For each k , let h_k be the ordered concatenation of $C_1, \dots, C_k$ using the same fixed internal orders. Define

$$\sigma_k = 1 \iff G(h_k) \in R_k.$$

Freshness gives $G(h_k) \notin P_k$. Hence if $\sigma_k = 0$, the output at h_k is outside $L_k = P_k \cup R_k$.

For a fixed target L_i , extend h_i by the private remainder R_i to obtain one complete L_i -text. At every earlier boundary h_k with $\sigma_k = 1$, the output belongs to R_k and is invalid for L_i . At its own boundary h_i , the output is invalid if $\sigma_i = 0$. Thus this one text witnesses all $m_i(\sigma)$ charged mistakes, at exactly the observed counts in $E_i(\sigma)$. The resulting mistake and convergence time lower bounds are (7); other errors made by G can only increase those quantities.

The infinite concatenation of all finite blocks is used only to specify a consistent family of finite histories. It is not asserted to be a text of any one target. Each lower-bound argument above fixes i and uses the legal prefix h_i followed by a legal continuation for that same target.

Optimality of every schedule. The lower bound shows that the schedules suffice to find all optimal guarantees. We must still prove that every schedule is itself optimal. First, no two distinct schedules have coordinatewise ordered mistake vectors. Suppose otherwise that $m_i(\tau) \leq m_i(\sigma)$ for every i , and let k be the first index where $\tau_k \neq \sigma_k$. The inequality at $i = k$ forces $\tau_k = 1$ and $\sigma_k = 0$. At $i = k + 1$, the difference of the mistake counts is

$$m_{k+1}(\tau) - m_{k+1}(\sigma) = 1 - (\tau_{k+1} - \sigma_{k+1}).$$

For this to be nonpositive, we must also have $\tau_{k+1} = 1$ and $\sigma_{k+1} = 0$. But then

$$m_{k+2}(\tau) - m_{k+2}(\sigma) = 2 - (\tau_{k+2} - \sigma_{k+2}) \geq 1,$$

a contradiction. Therefore $\tau = \sigma$ whenever all the mistake inequalities hold.

Now suppose an arbitrary G dominates G_σ . The reduction just proved supplies a schedule τ with $m_i(\tau) \leq M_i(G) \leq m_i(\sigma)$ for every i , so $\tau = \sigma$. Applying the convergence time part of the same reduction gives $d_i(\sigma) \leq D_i(G) \leq d_i(\sigma)$, while the mistake inequalities also become equalities. No coordinate can improve strictly. Thus every schedule profile is Pareto minimal. Conversely, every Pareto-minimal generator must have the same profile as its dominating schedule, proving the final assertion. $\square$

The two constant schedules illustrate the tradeoff. Always choosing C_{k+1} makes no mistakes at the earlier decision points for target L_i , but makes one at P_i . Its guarantees are therefore $M_i = 1$ and $D_i = s_i + 1$. Always choosing R_k avoids the mistake at P_i but makes one at each earlier decision point, giving $M_i = i - 1$ and $D_i = s_{i-1} + 1$ for $i \geq 2$, with $M_1 = D_1 = 0$. Theorem A.4.1 determines all intermediate choices as well. Its lower bound applies to every deterministic generator, and its minimality statement shows that each listed profile is an optimal tradeoff across the targets, completing the special-case characterization.

References

- Moses Charikar and Chirag Pabbaraju. Pareto-optimal non-uniform language generation. In *Proceedings of the International Conference on Algorithmic Learning Theory*, 2026. URL <https://proceedings.mlr.press/v313/charikar26a.html>.
- Jon Kleinberg, Anay Mehrotra, Amin Saberi, and Grigoris Velegkas. On language generation in the limit with bounded memory. *arXiv preprint arXiv:2605.30324*, 2026a. URL <https://arxiv.org/abs/2605.30324v1>.
- Jon Kleinberg, Charlotte Peale, and Omer Reingold. Mistake-bounded language generation. *arXiv preprint arXiv:2605.10809*, 2026b. URL <https://arxiv.org/abs/2605.10809v1>.
- Giorgio Racca, Michal Valko, and Amartya Sanyal. Language generation with replay: A learning-theoretic view of model collapse. *arXiv preprint arXiv:2603.11784*, 2026. URL <https://arxiv.org/abs/2603.11784v2>.